

CloudLearn Full Simulator Architecture

Local-first cloud learning platform with AWS-like workflows, runtime bundles, certification, and persistent state

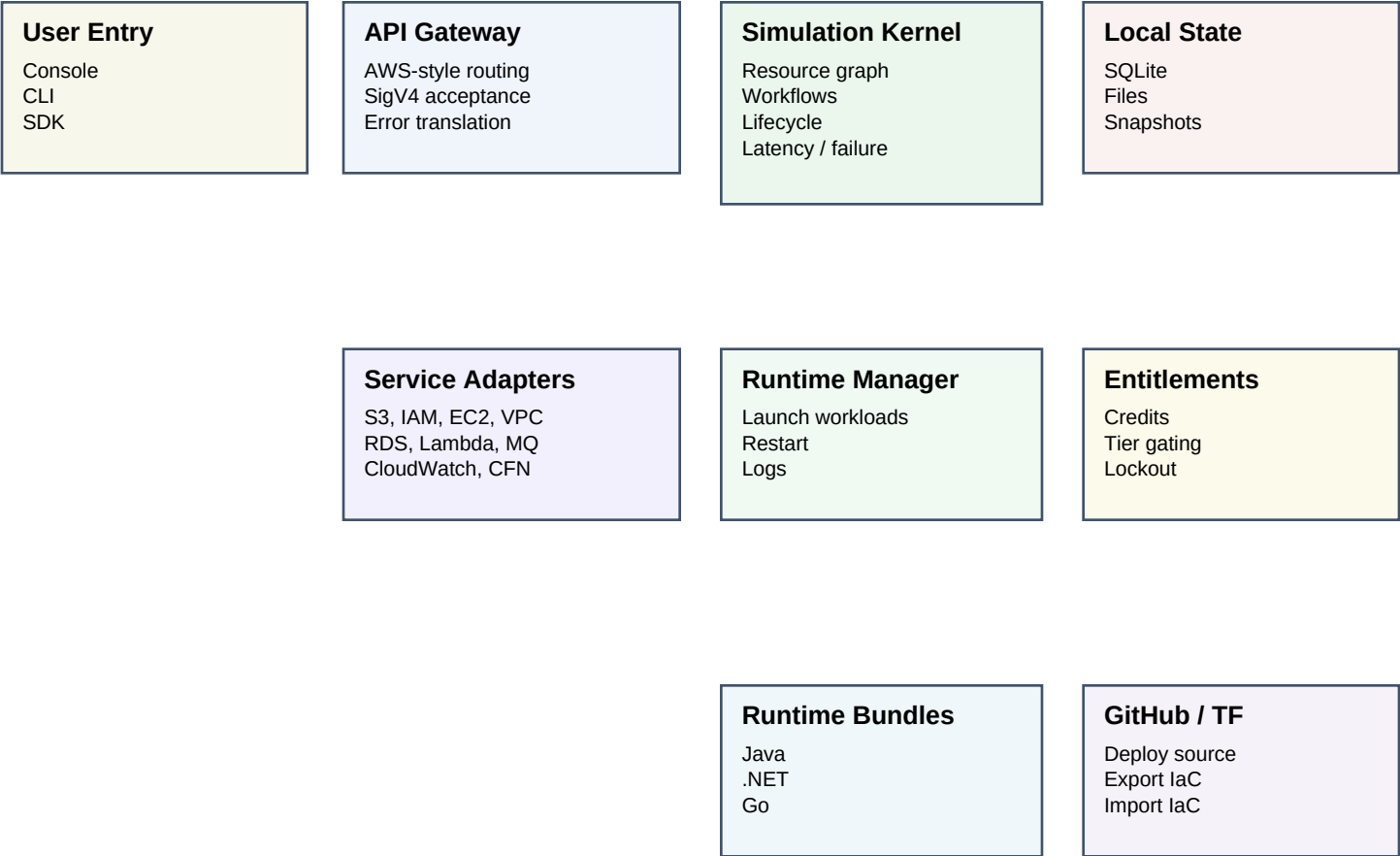
CloudLearn is a simulator and training platform that behaves like a cloud provider from the user's point of view while staying lightweight and fully local. It uses a custom simulation core, AWS-compatible adapters, and pluggable runtimes instead of recreating AWS internals.

Product goals

- Run locally on macOS, Windows, and Linux.
- Persist workflow state so users can stop and restart.
- Expose common AWS APIs and console-style flows.
- Drive every UI action through the documented API surface.
- Provide turnkey runtimes for Java, .NET, Go, PHP, and Python.
- Support certification exercises and scoring.
- Export and import infrastructure through Terraform.
- Simulate VPC, AZ, and security group behavior with lightweight engines.

Top-Level Runtime Stack

The simulator is a local control plane that orchestrates adapters, persistence, entitlementment, and workload runtimes.



Platform Services

The control plane adds the product features around the simulator core.

Control plane responsibilities

- Own simulator startup and shutdown.
- Route incoming API and UI actions to the right service adapter.
- Store all durable state locally.
- Issue entitlement and credit decisions.
- Track certification attempts and completions.
- Connect GitHub sources to runtime bundles.
- Keep service implementations thin by reusing shared engines.

Security and licensing

- Sign licenses, bundles, and exercise packs.
- Detect tampering and quarantine the client.
- Require cloud-issued unlock for blocked devices.
- Cache entitlements offline when allowed.
- Keep UI thin; enforce in backend services.

Persistence model

- SQLite for structured state and workflow tables.
- Local filesystem for uploads, logs, and artifacts.
- Event log for replay and audit.
- Snapshots for fast stop/start recovery.
- Encrypted token store for licenses and GitHub access.

Runtime and Deployment

The runtime layer turns source code or artifacts into local, cloud-like workloads.

Runtime Manager

Select bundle
Prepare workload
Start / stop / restart

Runtime Bundles

Language-specific packaging
Startup templates
Health checks

Local Host

Container engine or sandbox
Ports
Volumes
Logs

Supported runtime bundles

- Java bundle: JDK, Maven/Gradle hooks, Spring Boot / Micronaut / Quarkus-friendly startup.
- .NET bundle: dotnet run or published artifact startup.
- Go bundle: compile to static binary and run minimal container.
- PHP bundle: PHP-FPM or built-in server mode.
- Python bundle: uv, pip, virtualenv, uvicorn/gunicorn support.

GitHub deployment

- GitHub App or scoped OAuth tokens.
- Browse repo, branch, tag, and commit.
- Build locally and deploy into the selected runtime.
- Track deployment history and rollback.

Exercises, Validation, and Portability

This is where the platform becomes a learning product rather than a raw simulator.

Certification Engine

Labs, scoring, hints, time limits
Local progress tracking
Hidden validation rules

AWS Validation Mode

Run same workflow against AWS
Compare responses and state
Optional and not default

Terraform Bridge

Export simulator state to IaC
Import IaC into simulator
Keep desired state aligned

Plan / credit gating

- Free, Pro, Max, and Enterprise tiers.
- Credits unlock advanced services and exercises.
- Capability registry decides what is available.
- Entitlement changes are enforced in backend services.

Installer and distribution

- Native installers for macOS, Windows, and Linux.
- Signed packages and update channels.
- Background service or tray app if desired.
- Local data directories and state initialization.
- Optional Docker Compose stack for fast local launch.

Docker Compose Mode

Compose brings up the control plane, gateway, runtime manager, and persistence volume.
Use it for demos, onboarding, and

Service Catalog View

The simulator should present a catalog of cloud capabilities rather than expose every internal primitive.

Storage

S3
EBS-like volumes
EFS-like shares

Compute

EC2
Auto Scaling
Batch

Networking

VPC
Route 53
API Gateway

Identity

IAM
identity personas
session roles

Data

RDS
NoSQL
cache

Integration

Lambda
SQS
SNS

Observability

CloudWatch
traces
metrics

Infrastructure

CloudFormation
Terraform bridge
drift checks

Containers

ECR
ECS
EKS-aligned flows

The service catalog should also attach credit cost, entitlement tier, and certification support to every capability.

Detailed Service Architecture 1

S3, IAM, and EC2 are the first services that define the simulator's daily workflow.

S3

Back end: filesystem or embedded object store.
Model: buckets, objects, tags, versions, multipart uploads.
Behaviors: create, upload,

IAM

Back end: simplified evaluator.
Model: users, roles, groups, policies, sessions.
Behaviors: allow/deny, personas, role assumption, gating.

EC2

Back end: container or sandbox runtime.
Model: instances, metadata, storage, security groups.
Behaviors: pending, running,

EC2 runtime mapping

- Target OS and runtime stack become a container template.
- AMI maps to an image plus startup and metadata config.
- EBS maps to local volumes or bind mounts.
- Instance metadata service is provided locally.
- Console output and boot scripts are surfaced to the UI.

Detailed Service Architecture 2

VPC, RDS, and Lambda cover network isolation, data services, and code execution.

VPC

Back end: routing and policy simulation.
Model: VPC, subnet, route table, security group, endpoint.
Behaviors: ingress/egress,

RDS

Back end: local DB or containerized database.
Model: instance, parameter group, subnet group, snapshot.
Behaviors: create, start, stop,

Lambda

Back end: sandboxed process or container.
Model: function, handler, timeout, memory, triggers.
Behaviors: invoke, log, timeout,

These services should be lightweight but workflow-compatible. The user should recognize the control flow even if the i

Detailed Service Architecture 3

Queues, messages, logs, and template deployment complete the basic cloud workflow loop.

SQS / SNS

Back end: in-memory or local broker.
Model: queues, topics, subscriptions, messages.
Behaviors: enqueue, receive,

CloudWatch

Back end: event and metric store.
Model: logs, streams, metrics, alarms, dashboards.
Behaviors: query, publish, alert, display health.

CloudFormation

Back end: template parser and resource graph applier.
Model: stacks, changesets, outputs, events.
Behaviors: create, update,

These services are the glue for certification labs and app workflows. They should drive state changes in the same kernel.

Detailed Service Architecture 4

Container and edge services support packaged workloads and local routing.

Containers

Back end: Podman or Docker engine integration.
Model: image, repo, task, service, cluster, deployment.
Behaviors: push, pull, run,

API Gateway / Edge

Back end: local gateway router.
Model: APIs, routes, stages, custom domains, DNS records.
Behaviors: route traffic, deploy stage, expose endpoints.

Route 53

Back end: local name resolution map.
Model: hosted zones, records, aliases.
Behaviors: resolve simulator

Provider expansion path

- AWS first, because the market and training demand are strongest.
- Reuse the same kernel for Azure and GCP through provider adapters.
- Keep console skins, API facades, and terminology mappings separate from the kernel.
- Preserve the same workflow engine and local persistence model.

Non-goals

- Do not recreate every AWS internal implementation detail.
- Do not require cloud hosting for the simulator.
- Do not make the UI or frontend enforce licensing.
- Do not simulate a real hypervisor when a container is enough.

Master Layered Architecture

This page captures the complete stack used for design and implementation decisions.

1. Experience Layer

Console, CLI, SDK, labs, and certification views all call the same

Governance

Credits, tiers, lockout, and signed

2. API Contract Layer

Each service exposes request/response shapes, errors, state

Certification

Scenario packs, grading, hints, and

3. Routing Layer

Resolves service, region, account, resource, and entitlement

GitHub Deploy

Connect repos and deploy source into

4. Simulation Kernel

Owns the resource graph, workflow engine, lifecycle transitions,

Terraform

Import and export desired

5. Shared Engines

Resource graph, lifecycle, policy, topology, runtime, event, and

Validation Mode

Compare simulator behavior with real

6. Service Adapter Layer

S3, IAM, EC2, VPC, RDS, Lambda, SQS/SNS, CloudWatch,

Security

Signing, tamper detection, encryption,

7. Runtime Bundle Layer

Java, .NET, Go, PHP, and Python bundles provide turnkey local

Provider Expansion

AWS first, then Azure and GCP through

8. Local Host Layer

Containers, sandboxes, ports, volumes, metadata endpoints, and

Docker Compose

Supported local launch path with

9. Persistence Layer

SQLite, artifacts, encrypted tokens, audit history, event logs,

Master rule

- Keep the user experience cloud-like.
- Keep the implementation lightweight.
- Keep the core provider-neutral.
- Keep state local and restartable.
- Keep premium features governed by entitlements.

Capability Packs and Lazy Activation

Capabilities are delivered as signed packs that are downloaded and activated on demand.

Pack types

- Service packs: S3, IAM, EC2, VPC, Lambda, RDS, queues, etc. from remote registry.
- Runtime packs: Java, .NET, Go, PHP, Python.
- Exercise packs: guided labs, scoring, hidden checks.
- Provider packs: AWS first, then Azure and GCP.

Pack lifecycle

- Discoverability.
- Check entitlement and credits.
- Download the signed artifact.
- Verify signature and compatibility.
- Activate locally and cache for offline reuse.
- Update or roll back by version.

Pack structure

manifest.json, adapter code, API schemas, UI metadata, state model, tests, fixtures, and signature data.

The pack should be versioned and can include optional runtime hooks or container references.

A pack becomes visible in the catalog immediately, but its code only loads when a user or workflow needs it.

Pack Admission and Cloud-Agnostic Design

A pack must speak the simulator API contract and remain cloud-neutral in its core logic.

Pack admission rules

A pack must provide documented actions, request schemas, response schemas, error mappings, state transitions, and region behavior where relevant.

Admission checks include signature validation, version compatibility, entitlement validation, API contract validation, schema validation, and optional contract tests.

Cloud-agnostic pack model

The core capability logic should be provider-neutral. AWS, Azure, and GCP become provider adapters and profiles layered on top of the same capability model.

The core owns resource behavior, lifecycle, workflow logic, persistence shape, and events. Provider adapters own API names, terminology, and request/response mapping.

MVP rule

- Reject packs without AWS-like API support.
- Reject packs without valid schemas or transitions.
- Reject packs that hardcode provider behavior into the core capability logic.
- Require AWS first, but define Azure and GCP adapter slots from day one.

MVP Scope

The MVP should be small, fast to install, and still feel like a real cloud workflow simulator.

MVP includes

Local control plane, AWS-style API-driven actioning, durable local state, Docker Compose deployment, free tier support, signed entitlement flow, capability packs, lazy activation, S3, IAM basics, EC2 basics backed by a local container runtime, VPC basics, one runtime bundle path, GitHub deploy, basic certification exercises, and tier/credit gating.

MVP defers

- Full service catalog depth.
- RDS.
- Lambda.
- SQS / SNS.
- CloudWatch.
- CloudFormation.
- AWS validation mode.
- Terraform import/export at scale.
- Azure and GCP provider packs.
- Advanced multi-region behavior.

MVP success criteria

Users can sign up for Free.
Install or start with Docker Compose.
Create and manage S3 buckets.
Launch a simple EC2-like workload locally.
Run a basic lab and resume after restart.
Activate packs lazily on first use.